

The effects of AI recommendation on epistemic diversity

I describe a simple model (sections 1–3), make some guesses about how the model may behave (section 4), and discuss some potentially philosophically interesting consequences of these conjectures (sections 5–6). The model comes with a small Python implementation, which I outline before turning to the model itself.

Implementation

The model is realised as a small, self-contained Python package, `model/`, with one module per modelling primitive. `BernoulliBandit` holds the two arms and their Bernoulli means and enforces $p_A > p_B$. `BetaBernoulliAgent` is a single agent: it keeps one Beta posterior per arm, chooses by posterior mean, and performs the conjugate update on each observed payoff. `Recommender` is the interface for the AI of sections 3–5; in this phase the only concrete implementation is `NullRecommender`, which abstains, so the simulation reduces to the unaided regime. `discovery.py` collects the operational definitions of the discovery event D_i (section 1), and `run_simulation` is the orchestrator.

The simulation is a single loop. A community of n agents faces the shared bandit for T stages; at each stage every agent (i) optionally receives a recommendation, (ii) pulls the arm with the higher posterior expected payoff, (iii) observes a Bernoulli payoff, and (iv) updates the posterior of the pulled arm. Each agent draws from its own random stream, spawned from one master seed via `SeedSequence.spawn`, so the agents are statistically independent yet the run as a whole is reproducible. The accompanying notebook reproduces the unaided results of section 2; the recommender of section 3 onward is wired into the interface but not yet behaviourally implemented.

1 Standard two-armed bandit

Start with a standard two-armed bandit model. There are two arms, A and B . Arm A has mean payoff p_A . Arm B has mean payoff p_B . Assume $p_A > p_B$, so A is the better arm. We take the payoffs to be Bernoulli: pulling an arm returns 1 with probability equal to that arm’s mean and

0 otherwise. This is the natural choice here, because Bernoulli payoffs are conjugate to the Beta beliefs the agents carry below, so each observation updates a belief in closed form.

There are n agents. Assume for now that agents do not communicate or observe others' payoffs. Each agent carries two independent Beta beliefs, one over p_A and one over p_B ; unless stated otherwise these start uniform, $\text{Beta}(1, 1)$, and the same prior is shared across the community. Because the payoffs are Bernoulli, an agent conditions on an observed payoff by the conjugate update: a success increments that arm's α , a failure its β . Each agent conditions only on its own payoffs.

Every agent is a myopic Bayesian. At every stage it pulls the arm with the higher posterior expected payoff, that is, the arm with the larger posterior mean $\alpha_i/(\alpha_i + \beta_i)$, breaking ties uniformly at random.

Let D_i be the event that agent i discovers which is the better arm. We make this precise through two operational criteria, both evaluated on the agent's state at the end of the run and both implemented in `discovery.py`:

1. *Last- k* : the agent's last k pulls are all of arm A , so it has, in effect, settled on the better arm.
2. *Posterior gap*: the agent's posterior means separate in the right direction by at least a margin δ , that is $E[p_A | h] - E[p_B | h] \geq \delta$.

The two criteria need not agree on any single run, but they coincide with high probability once the horizon is long: an agent that has pulled A for many consecutive steps has typically also built up a posterior-mean gap in A 's favour, and conversely. We therefore treat D_i as well defined up to this asymptotic equivalence and report results under both criteria.

The community succeeds (D) if at least one agent succeeds, so

$$D = D_1 \cup \dots \cup D_n.$$

2 Unaided collective success

I use P for the “objective” probabilities determined by the model specifications. Write $q = P(D_i)$ for the individual discovery probability, which by symmetry is the same for every agent.

What fixes q . Under this objective reading q is not a free parameter; it is fixed by four ingredients: the gap between the arms, the horizon, the prior, and the discovery criterion. The gap is the primary driver. Writing $\Delta = p_A - p_B$, when Δ is large the arms are easy to tell apart and q is high, whereas as $\Delta \rightarrow 0$ they become statistically indistinguishable and q falls toward the level a non-discovering agent would reach by chance. The horizon T controls how much evidence an agent can gather, so q increases with T , though (as discussed below) it does not approach 1. The criterion enters through its tolerance: the window k for last- k and the margin δ for the posterior

gap, with a looser criterion (smaller k or smaller δ) counting more agents as successful and so raising q . Rather than postulate a value for q , then, we read it off the model for a given gap, horizon, prior, and criterion.

The prior. It is worth being explicit about how the simulation initialises the beliefs, since the prior sets each agent’s starting point and its early inertia. Every agent begins each arm at a uniform Beta(1, 1) belief, and all n agents in a community are given the same prior. A Beta(1, 1) is flat on $[0, 1]$: it expresses no preference between the arms, so both posterior means start at $\frac{1}{2}$ and the first pull is a coin flip, and it lets the earliest observations move a belief a long way, since after a successes and b failures on an arm the posterior mean is $(a + 1)/(a + b + 2)$. A more concentrated prior Beta(α_0, β_0) with larger $\alpha_0 + \beta_0$ would damp those early swings and make an agent harder to divert from a good arm; an asymmetric prior would bias which arm an agent favours before seeing any data. We keep the flat, shared prior as a neutral baseline and treat (α_0, β_0) as a knob to revisit.

An analytic handle as $T \rightarrow \infty$. The myopic rule never explores for its own sake; it always pulls the arm it currently believes is better, and this makes the long-run behaviour tractable. If both arms were pulled infinitely often, their posterior means would converge to p_A and p_B ; since $p_A > p_B$ the agent would then prefer A from some point on, contradicting infinitely many B-pulls. So each agent pulls one arm only finitely often and the other forever: the play *absorbs* on a single arm. In this limit the last- k criterion becomes exactly the event that the agent absorbs on A, and the posterior-gap criterion becomes that same event together with the mild condition that the final gap clears δ ; to leading order both give

$$q = P(\text{the agent settles on } A).$$

This probability lies strictly between 0 and 1. It is below 1 because of *incomplete learning*: an unlucky early run on A can pull its posterior mean below that of B, so the agent switches to B; if by then A’s mean has fallen below p_B , then once B’s posterior mean settles near p_B it stays above A’s frozen value forever, and the agent never samples A again or collects the evidence that would correct it. It is above 0 because a good early run locks the agent onto A in the same way. There is no elementary closed form at general (p_A, p_B) : writing $f(\alpha_A, \beta_A, \alpha_B, \beta_B)$ for the probability of eventually settling on A from a given pair of Beta states, $q = f(\alpha_0, \beta_0, \alpha_0, \beta_0)$ solves a fixed-point recursion over those states, which the Phase 1 simulation evaluates by Monte Carlo rather than in closed form. What is clear analytically is the shape: q increases with the gap Δ , it stays strictly below 1 for every finite gap while approaching 1 as the gap widens, and the prior enters only through the common starting state. This ceiling below 1 is what makes the community question non-trivial; if individuals discovered A almost surely there would be nothing for the AI to spoil.

Simulation: calibrating q . Two short simulation studies make this concrete and were used to choose parameters; both run the unaided model with n independent agents and report $\hat{q}$ as the fraction of agents satisfying each criterion at the end of the run. The notebooks are experiments/1b. Phase 1 – Gap-to- q Calibration.ipynb (the gap sweep) and experiments/1c. Phase 1 – Prior-to- q Calibration.ipynb (the prior sweep).

First, the gap alone. Centering the arms at $\frac{1}{2}$ (so $p_A = \frac{1}{2} + \Delta/2$ and $p_B = \frac{1}{2} - \Delta/2$), fixing the flat Beta(1, 1) prior and $T = 200$, and sweeping Δ gives the table below. Both criteria increase with Δ , confirming that the gap is the primary driver, yet neither comes near 0.05: as $\Delta \rightarrow 0$ they level off at a floor close to $\frac{1}{2}$. This is the absorption picture above. With a vanishing gap the agent still settles on one arm, but which arm is essentially a coin flip, so “settled on A” has probability $\approx \frac{1}{2}$. The gap on its own cannot manufacture a rare-discovery regime.

Gap sweep	(Beta(1, 1) prior, $T = 200$, $n = 1500$ per gap; notebook 1b)					
Δ	0.002	0.01	0.05	0.10	0.20	0.40
last- k ($k = 20$)	0.52	0.50	0.58	0.68	0.79	0.96
posterior gap ($\delta = 0.05$)	0.48	0.47	0.55	0.65	0.79	0.96

Second, the prior. Holding $\Delta = 0.01$ and $T = 200$ and instead concentrating the shared prior to Beta(s, s) gives the second table, and the two criteria part company. The posterior-gap probability collapses as s grows: a tight prior pins both posteriors near $\frac{1}{2}$, the belief gap rarely clears δ , and $\hat{q}_{\text{gap}}$ crosses the target at $s \approx 55$, that is, a Beta(55, 55) prior. The last- k probability barely moves, sitting near $\frac{1}{2}$ throughout: it records which arm is pulled, not how far beliefs have separated, and with the arms this close which arm the agent settles on is essentially a coin flip whatever s is. So the practical knob for a target q under the posterior-gap criterion is the prior concentration (or, equivalently, the horizon), not the gap; and the two criteria, asymptotically equivalent as $T \rightarrow \infty$, sit far apart in this finite-horizon, strong-prior regime. Making last- k itself rare would instead require shortening the horizon toward k .

Prior sweep	(fixed $\Delta = 0.01$, $T = 200$, $n = 1500$; notebook 1c)					
s	1	10	50	100	150	200
last- k ($k = 20$)	0.53	0.51	0.50	0.50	0.49	0.48
posterior gap ($\delta = 0.05$)	0.49	0.33	0.06	0.01	0.00	0.00

The community baseline. To fix ideas, suppose $q = 0.05$, which (by the prior sweep) the posterior-gap criterion reaches at this gap and horizon once the shared prior is concentrated to about Beta(55, 55). In that regime no individual agent is at all likely to succeed. Because the agents are independent, the probability that the community succeeds is nonetheless much higher than the probability of individual success. With $n = 100$ agents,

$$P(D) = 1 - (1 - q)^n = 1 - 0.95^{100} \approx 0.994.$$

This is the unaided baseline, and it is what the simulation produces: the measured fraction of communities in which at least one agent discovers A agrees with the closed form $1 - (1 - \hat{q})^n$, because with no recommender the agents share no common state and so remain genuinely independent.

3 AI recommender

Now introduce an AI recommender. At each stage t , agent i gives the recommender her private history h_i^t (which arms she has pulled and which payoffs she has received). By conditioning on that history, the agent also has posterior π_i^t . The recommender ρ takes these two things as inputs and returns a recommendation

$$R_i^t := \rho(h_i^t, \pi_i^t, Z) \in \{A, B\}.$$

Interface implemented in `model/recommender.py`: `abstract Recommender.recommend(agent_id, history, posterior_alpha_beta, rng)` returns an optional `Recommendation`(arm, likelihood). Phase 1 ships `NullRecommender`, which always returns `None` — i.e. the unaided regime. Phase 2 needs a concrete subclass.

The variable Z represents the features of the recommender that are common among all agents, e.g. the algorithm that ρ uses to make recommendations. Suppose (here is a big simplification) $Z \in \{G, F\}$, that is, Z can be in two states (a good state or a failure state). Suppose

$$P(Z = G) = 0.99, \quad P(Z = F) = 0.01.$$

Pending Phase 2 design: Z is drawn once per simulation (per community) inside the concrete recommender’s constructor, and is read by `recommend(...)` when producing each R_i^t . The shared `recommender_rng` already spawned in `run_simulation` is the natural place to draw it. We also need to record Z on the `SimulationResult` so per-trial conditioning ($P(D) | Z$) can be reported empirically.

Interpretive question about Z , still unresolved. The paper writes $P(Z = G) = 0.99$, which reads like an exogenous draw, but the prose says Z “represents the features of the recommender”, i.e. the algorithm’s design — something fixed, not random. Which is meant? Several readings are compatible with the maths:

1. *Per-community draw of recommender identity*: each community gets a fresh recommender whose hidden type is sampled $\{G, F\}$ once. This is what the current Phase 2 plan above assumes; it is what makes “community discovery” a non-trivial random variable over Z .
2. *Per-trial frequentist statement about a single fixed recommender*: Z is the operating mode of one recommender in production, G/F the fraction of the time it is well/poorly behaved. Then $P(Z = G)$ is a long-run rate, not a draw.

3. *Subjective uncertainty over a fixed-but-unknown recommender*: Z is the agent’s (or modeller’s) credence over the recommender’s type. Then $P(Z = G)$ is epistemic, not aleatoric.

The reading matters for the conditioning step in the next paragraph: under (1) the per-simulation correlation across agents (the §6 mechanism) follows automatically because Z is shared; under (3) the agent should arguably condition on R_i^t jointly with their evolving posterior *over* Z , not on a sampled Z value. And under (2) it is unclear whether the community-level calculation in §5 (which treats Z as constant across the n agents in a given community) is even well-posed — it would need the recommender’s mode to be constant across all n agents within one community, which is a stronger claim than “ F happens 1% of the time”. Pinning this down before Phase 2 implementation.

Interpretation 2 is what I had in mind originally.

In the aided regime, every agent remains a myopic Bayesian but conditions on the AI recommendation before deciding which arm to pull. So, in the aided regime, first an agent conditions on his history to get the posterior π_i^t . Then he conditions on R_i^t . Finally, he chooses the arm with the higher $\pi_i^t(\cdot | R_i^t)$ expected payoff.

Pending: `BetaBernoulliAgent.apply_recommendation` currently raises `NotImplementedError`. To implement it we need a likelihood $P(R | \text{better arm}, h, \pi, Z)$ on the `Recommendation.likelihood` field. The agent’s Bayes update on R then folds that likelihood into the per-arm Beta posterior. *Big modelling choice still open*: does the agent know $P(Z = G)$ and the per-state likelihoods? The paper’s setup suggests yes (so the conditioning is a closed-form Bayes update on Z first, then on R), but worth confirming.

I hadn’t thought about whether the agents know the value of $P(Z = G)$. Perhaps both options are worth investigating eventually? I don’t know. I suggest we start with whether is simplest.

4 Model conjectures

I’m guessing that recommenders and recommendation likelihoods can be defined so that, say,

$$P(D_i | Z = G) = 0.1, \quad P(D_i | Z = F) = 0.$$

Pending: pick a concrete recommendation rule ρ that realises these numbers. Candidate from §4: in G , R matches the better arm A with probability $1 - \eta_G$ (tunable); in F , $R = B$ with probability $1 - \eta_F$, concentrated at early stages where it bites hardest. Both η_G and η_F are then sweep parameters to calibrate against the targets 0.1 and ≈ 0 .

ρ doesn’t have to realize exactly these numbers. Again, just examples.

One idea is that in the good state, the recommender is somewhat helpful and often nudges the agent toward the better arm A , raising the probability of discovery from 0.05 to 0.1. In the failure condition, the recommender gives misleading B recommendations that prevent success almost

surely. Suppose, for example, that at early stages, when continued sampling of A is important, the recommender often recommends B . If there are enough early B recommendations, the agent won't discover that A is the good arm.

The “early B bias in F ” suggests ρ has time-dependent behaviour ($\eta_F = \eta_F(t)$), not just state-dependent. That is a non-trivial complication for the conditioning step in §3: the agent's likelihood must depend on t too. Worth deciding before Phase 2 implementation: do we want stationary or non-stationary ρ ?

I was thinking we shouldn't assume that ρ is stationary, that it may exhibit time-dependent behavior.

5 Individuals improve, the community gets worse

For all i , in the aided regime, the probability that i succeeds is

$$(0.99)(0.1) + (0.01)(0) = 0.099.$$

So for each individual the probability of discovery improves from 0.05 to 0.099.

The community calculation is different because Z is common across all agents. Conditional on $Z = G$, the agents still have independent private histories, and each has success probability 0.1. So the probability that at least one of the 100 agents succeeds is

$$1 - (1 - 0.1)^{100} \approx 0.99997.$$

Conditional on $Z = F$, no one succeeds.

So, the aided community discovery probability is

$$(0.99)[1 - (1 - 0.1)^{100}] + (0.01)(0) \approx 0.98997.$$

The unaided community success probability is about 0.994. So the AI raises every individual's probability of discovery while lowering the community's probability of discovery.

Phase 2 success criterion: a notebook 2. Phase 2 - Aided Bandit.ipynb that reports $\hat{q}_{\text{aided}} \approx 0.099$ and $\widehat{P(D)}_{\text{aided}} \approx 0.99$, conditioning explicitly on the recorded Z values per trial. Sweep η_G, η_F to show the headline result is robust, not knife-edge.

6 The mechanism

The mechanism is the introduction of dependence from the common AI recommender. In the unaided regime, some agents get unlucky and abandon A , but others continue sampling it. Their unluckiness is not correlated, so the community benefits from independent variation in payoff

histories.

In the aided regime, the agents still have private evidence, but their evidence now includes recommendations generated by the same AI. In the good condition, this common system helps many agents. In the rare failure condition, it moves many posteriors in the same wrong direction. In the aided regime unluckiness becomes correlated. That is the sense in which this AI recommendation can decrease epistemic diversity.

Finally, note that the model is not a wheel-shaped network with AI as the central node. The AI does not sample from the arms or generate any payoff history.

Phase 3 (the empirical verification of the mechanism): quantify the correlation explicitly. For each trial compute (a) the cross-agent correlation of $\hat{p}_A$ trajectories, (b) the conditional correlation given Z , and (c) the residual after removing the Z -mean. Predict: cross-agent correlation is ≈ 0 in the unaided regime, > 0 in the aided regime, and the gap is entirely attributable to Z .

7 Next steps

1. Fill in the details of the model and run simulations to confirm the conjectures and effects in sections 4 and 5.
2. Think about how to make the model more interesting. For example, we could introduce history-sharing networks together with AI recommendations, and we could model more sophisticated AI recommenders.

Concrete Phase 2 / Phase 3 punch list distilled from the comments above: (1) decide on the discovery definition (§1); (2) parameterise $(p_A, p_B, T, \text{prior})$ so unaided $\hat{q}$ matches the postulated 0.05 (§2); (3) pick the functional form of ρ and whether it is stationary in t (§4); (4) decide what the agent knows about Z and the per-state likelihoods (§3); (5) record Z on `SimulationResult` and report $\widehat{P(D)} \mid Z$ (§3); (6) verify the mechanism via cross-agent correlations (§6). Stretch goal (history-sharing networks) is genuinely Phase 4+: the existing `e_network_inequality` codebase will be the reference there.